IBM

IBM **Aspera**®

IBM Aspera on Cloud Automation Development Guide

Laurent Martin, IBM

2026/11/09



BUILD • AUTOMATE • ORCHESTRATE

Contents

1 Concepts **3**
- 1.1 Introduction . . . 3
  - 1.1.1 Where to start . . . 3
  - 1.1.2 Typographic conventions . . . 4
- 1.2 How AoC Automation works . . . 4
- 1.3 Terminology . . . 4
- 1.4 States and statuses . . . 5
  - 1.4.1 Workflow states . . . 5
  - 1.4.2 Workflow instance statuses . . . 5

2 Triggers **6**
- 2.1 Manual trigger . . . 6
- 2.2 Scheduled trigger . . . 6
- 2.3 Watch folder trigger . . . 6
- 2.4 Shared inbox trigger . . . 7
- 2.5 Delete folder trigger . . . 7
- 2.6 Trigger conditions and filtering . . . 7

3 Actions **9**
- 3.1 File transfer . . . 9
- 3.2 Package transfer . . . 10
- 3.3 REST API call . . . 11
- 3.4 Email notification . . . 11
- 3.5 Folder creation . . . 11
- 3.6 Custom variables . . . 12
- 3.7 Telestream quality control . . . 12
- 3.8 Action timeout defaults . . . 12
- 3.9 Payload rules enforced by the API . . . 13
  - 3.9.1 General rules . . . 13
  - 3.9.2 Location shapes . . . 13
  - 3.9.3 Action-specific rules . . . 13
  - 3.9.4 Outputs available to later steps . . . 14

4 Substitution variables **15**
- 4.1 Global workflow instance variables . . . 15
- 4.2 Action output references . . . 16
- 4.3 Path-based substitutions schema . . . 16

5 Managing workflows and jobs **17**
- 5.1 Workflow lifecycle . . . 17
  - 5.1.1 API access and pagination . . . 18
  - 5.1.2 Automatic deactivation after failures . . . 18
- 5.2 Job lifecycle . . . 18

6 Managing workflows with ascli **19**
- 6.1 CLI commands reference . . . 19
- 6.2 Exporting and importing workflows . . . 19
  - 6.2.1 Exporting a workflow . . . 19

6.2.2 Portable manifest format example . . . 19
6.2.3 Importing a workflow . . . 20
6.2.4 Creating a workflow step by step . . . 20
6.3 Finding the identifiers . . . 21

7 Configuring API callback **22**

8 Design recipes and patterns **23**
8.1 Watch folder to multi-destination transfer . . . 23
8.2 Shared inbox package to MAM webhook . . . 23
8.3 Scheduled sync with differential overwrite . . . 23
8.4 Shared inbox fan-out with archive, webhook, review package and summary . . . 25

9 Best practices and troubleshooting **27**
9.1 Practices checklist . . . 27
9.2 Common pitfalls and resolutions . . . 27

IBM Aspera on Cloud Automation Development Guide

Important

This guide covers workflow development and automated operations using the **IBM Aspera on Cloud (AoC) Automation API** and the **IBM Aspera CLI (`ascli`)**. It applies to IBM Aspera on Cloud Automation API 1.0.5 and higher.

Chapter 1

Concepts

This chapter explains the execution model of AoC Automation, its terminology, and the states of workflows and jobs.

1.1 Introduction

IBM Aspera on Cloud (AoC) Automation provides cloud-native workflow orchestration for fast, reliable, and secure data movement across hybrid and multi-cloud storage tiers.

It is designed for workflow authors, systems integrators, and AI assistants building automated file pipelines with IBM Aspera on Cloud.

> **Note**
>
> The Automation app is available on the **Standard Plus** and **Premium** editions of Aspera on Cloud. Concurrent job limits: **5** (Standard Plus) / **10** (Premium). Queue size limits: **5,000** (Standard Plus) / **10,000** (Premium). It is not enabled by default: an org admin must enable it under **Admin > Applications > Automation** and add members. A member of the Automation app can use all its features and see all its content, whatever their workspace membership.

Prerequisites, before a user can run workflows:

- The **Activity** app is enabled, and the node attached to Aspera on Cloud sends data to it: create a node configuration policy (**Nodes and storage > Node policies > Configuration policies**) with **AEJ logging** checked, and apply it to the node.
- A tethered node needs HSTS 4.0 or higher (Windows is not supported) and the complete tethering procedure, including monitoring and metering configuration.
- The IP address of the transfer node is in the firewall allowlist.

Reference documentation of the Automation API: IBM API Hub.

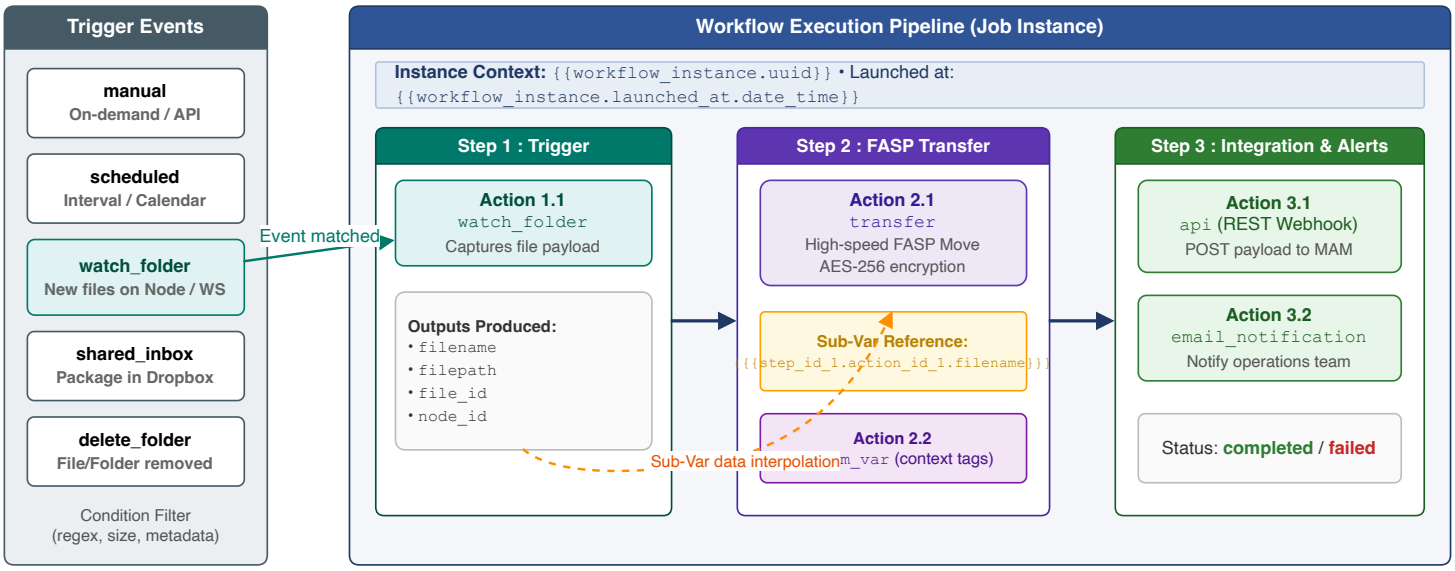
1.1.1 Where to start

Goal	Read
Understand the AoC execution model	How AoC Automation works, Terminology, States and statuses
Configure event and schedule triggers	Triggers, Trigger conditions and filtering
Configure transfers, API calls and actions	Actions
Write a payload the API accepts on the first try	Payload rules enforced by the API, Finding the identifiers
Pass data dynamically between steps	Substitution variables
Manage workflows from the command line	Managing workflows with ascli
Create a workflow from a file	Importing a workflow
Implement common end-to-end patterns	Design recipes and patterns
Check requirements before deployment	Practices checklist, Common pitfalls and resolutions

1.1.2 Typographic conventions

Format	Use	Example
Bold	A label of the web interface: button, menu, tab, field.	**Run Now**, **Active**, **Save**
Code	A literal of a file or code: attribute, status, action type, command, JSON/YAML fragment.	`watch_folder`, `transfer`, `completed`
Italic	A term where it is defined, or a workflow/step name chosen by the author.	*Ingest pipeline*, *Archive Drop*
Double quotes	A message or text that AoC displays.	”Package received”
`<PLACEHOLDER>`	A value that the reader replaces.	`<WORKFLOW_ID>`

1.2 How AoC Automation works



AoC Automation execution model: Trigger events, sequential Steps and Actions, and Sub-Vars data flow

A workflow is an ordered sequence composed of **Steps**, each containing one or more **Actions**:

1. **Definition**: An administrator or user defines a workflow with a single **Trigger** followed by one or more sequential **Steps**.
2. **Step Execution**: Steps run sequentially in the order defined by `step_order`.
3. **Action Execution**: Within each step, the actions run in parallel; `action_order` only gives their display order.
4. **Data Propagation**: Actions pass information downstream using mustache-like **Substitution Variables** (`Sub-Vars`), allowing dynamic destination paths, file names, API payloads, or email bodies.
5. **Instance Lifecycle**: When a trigger condition is met (or when triggered manually), AoC creates a **Workflow Instance** (Job) tracked in the Job Queue.

1.3 Terminology

Term	Meaning	In the web interface
Workflow	The definition of an automated business process. Identified by a numeric ID. Contains any number of steps.	Workflow
Workflow Instance	A single execution run of an active workflow (a Job).	Job
Step	A logical stage containing one or more ordered actions. Steps run sequentially; actions within a step run in parallel.	Step
Action	An individual operational task: Transfer, Send a Package, Call an API, Notify, Custom Variables, Create a Folder.	Action

Term	Meaning	In the web interface
Trigger	The first action of Step 1 that defines the conditions for launching a workflow instance.	Trigger
Sub-Var (Substitution Variable)	A dynamic placeholder resolved at runtime from prior action outputs or global workflow context.	Variable
Job Queue	The list of running and queued jobs. Beyond the concurrent maximum, jobs queue in FIFO order.	Job Queue
Permissions	Sharing rules on a workflow, for a user: `manage_jobs` (view, start, cancel, delete its jobs), `duplicate` (copy it) and `edit` (change it). The owner has all three.	Share

1.4 States and statuses

1.4.1 Workflow states

- `draft`: The workflow is under configuration. Automatic triggers are disabled.
- `validating`: Transitional state after `activate` on a `draft` workflow: Automation requests the permissions on the nodes. The workflow becomes `active` when they are granted, and returns to `draft` otherwise. It cannot be edited meanwhile.
- `inactive`: The workflow configuration is completed but execution is paused.
- `active`: The workflow is live. Automatic triggers spawn jobs when triggered.

1.4.2 Workflow instance statuses

The status names below are those of the Automation API; the web interface label is in parentheses.

- `initiating` (”Initiating”): The job is being started.
- `queued` (”Queued”): The job is waiting to run because the concurrent maximum has been reached.
- `queue_full`: The queue is full.
- `executing` or `resumed` (”Running”): The job is currently executing steps and actions.
- `paused` (”Paused”).
- `complete` (”Completed”): All steps and actions finished successfully.
- `failed` (”Failed”): An action encountered an unrecoverable error (transfer failure, for example).
- `timeout` (”Timed out”): An action did not complete within its timeout.
- `canceled` (”Cancelled”): The instance was aborted by a user or administrator.
- `deleted`: The job was deleted. It is no longer listed.

The workflow itself also has a status, which results from the validation of its permissions on the nodes: `not_started` (initial), `submitted` (permissions requested), `success` (granted) or `error`.

In the Automation API, a workflow instance also exposes `current_status` (human-readable description), `queue_position` (when `queued`), `workflow_trigger_type`, `action_id` (current action) and `action_statuses` (status and outputs of each action, per step). The `status` query parameter of `GET /workflow_instances` accepts any of the status names above. A job is identified by its numeric `id` or by its `instance_id` (UUID), in the path of the requests on a job.

Chapter 2

Triggers

Every workflow requires exactly one trigger defined as the first action of the first step.

2.1 Manual trigger

- Type: `manual`
- Trigger condition: Triggered on-demand via the Web UI (**Run Now**) or through the REST API (`POST /workflows/:id/launch`) and CLI (`ascli aoc automation workflows launch <id>`). The workflow must be `active`.
- Outputs: `workflow_instance.uuid`, `workflow_instance.launched_at.*`.
- **Constraint**: A workflow with a manual trigger must contain at least one `transfer` or `package_transfer` action.

2.2 Scheduled trigger

- Type: `scheduled`
- **Constraint**: A workflow with a time-based trigger must contain at least one `transfer` or `package_transfer` action.
- Configuration parameters:
  - `repeat_interval.repeat_type`: `minutes`, `hours`, `days`, `weeks`, `months`.
  - `repeat_interval.repeat_number`: Interval count (e.g., every 2 hours), an integer. Limits enforced by the API: `minutes` greater than 4, `hours` 1–24, `days` 1–7, `weeks` 1–4, `months` 1–12.
  - `repeat_interval.days_of_the_week`: Object of boolean flags (`monday` through `sunday`), required for `weeks`.
  - `repeat_interval.day_of_the_month`: Integer (1–31), required for `months`.
  - `start_date`, `end_date`: ISO 8601 timestamps bounding the schedule. Both must be in the future, and the start must precede `end_date`, which is optional.
  - A `scheduled` workflow that already has a running job cannot be launched manually.

> **Note**
>
> The recurrence interval begins when the **previous run completes**, not at the scheduled start time. For example: start at 01:00, repeat every 6 h, run takes 2 h → next run starts before 09:00 (3:00 AM + 6 h).

2.3 Watch folder trigger

- Type: `watch_folder` (labeled **File addition event** in the UI)
- Monitored location:
  - `source.type`: `node` (requires node secret) or `workspace`.
  - `source.node_id` or `source.workspace_id`.
  - `source.file_id`: Root folder ID to monitor.
  - `source.paths`: Monitored sub-paths; at least one item is required (`[{source: "/"}]`).

  - Through the API, `source.node_name` and `source.file_name` are also required: see Payload rules enforced by the API.
  - With a workspace, the list contains the workspaces where you are a member; an org admin can also browse all shared folders of the workspace. To monitor a workspace where you are not a member, select the folder through the node.
- `recursive_trigger`: Boolean (**Include sub folders**) — also trigger on files added to sub-folders.
- Condition filter (`conditions_type`): `all_new` (**Any file is added**, default) or `custom` (name, extension or size conditions, see Trigger conditions and filtering).
- This trigger has no archive settings: archiving or deleting the source after the transfer is configured on the `transfer` action (`source_archive`).

2.4 Shared inbox trigger

- Type: `shared_inbox` (labeled **Shared inbox event** in the UI)
- Monitored location:
  - `source.type`: `workspace`.
  - `source.workspace_id` and `source.workspace_name`: Workspace of the shared inbox.
  - `source.inbox_id` and `source.inbox_name`: Identifier and name of the Aspera on Cloud Shared Inbox (Dropbox). The identifier is the `dropbox_id` returned by `packages shared_inboxes list`; the property `dropbox_id` itself is rejected in `source`.
  - An org admin can choose to view the workspaces as **Admin** or as **User**.
- Condition filter (`conditions_type`): `all_new` (**Any file is added**, default) or `custom` — one or more metadata field conditions (field, operator, value). `custom` is only available when the inbox has usable metadata fields.
- Metadata extraction: Outputs package metadata fields filled by the package sender (accessible as Sub-Vars in downstream actions).

2.5 Delete folder trigger

- Type: `delete_folder` (labeled **File or folder deletion event** in the UI)
- Configuration:
  - `source`: Monitored folder, on a node or in a workspace (same selection as the watch folder trigger).
  - `conditions_type`: `all_new` (**This folder is deleted**) or `custom` (conditions on `file_name`, `folder_name`, `extension_name`).
  - With `all_new`: `trigger_on_file_deletes` (**Include top level files**) and `trigger_on_folder_deletes` (**Include top level sub-folders**) also trigger the workflow when a top-level file, or sub-folder, of the monitored folder is deleted.
  - `source.access_key`: Node access key for delete event monitoring. The API requires it, together with `source.node_id`, `source.file_id` and `source.paths`.

2.6 Trigger conditions and filtering

Watch folder and shared inbox triggers support granular condition lists:

```yaml
conditions_type: custom          # all_new (any file added) | custom
conditions:
  - field_name: "file_name"      # file_name | extension_name | file_size (watch folder); file_name |
↪  folder_name | extension_name (delete folder)
    field_type: "string"
    match_type: "regex"          # includes | excludes | matches | regex
    value: ".*\\.mp4$"
  - field_name: "file_size"
    field_type: "byte"
    match_type: "greater_than"   # greater_than_or_equal | less_than_or_equal | greater_than |
↪  less_than | equal
    value: 1048576               # bytes (> 1 MB)
join_type: "and"                 # and | or (default or for watch folder and delete folder triggers)
```

For the shared inbox trigger, conditions apply to the metadata fields of the inbox: `value` is a list, and the web interface always sets `join_type: and`. Date metadata fields use `on`, `after`, `before`, `on_or_before`, `on_or_after`; list and checkbox fields use `matches`, `does_not_match`.

Chapter 3

Actions

Each action step performs one specific task. Multiple actions can run **in parallel** within the same step (click the **+** icon below an existing action in the UI). Available action types: `transfer`, `package_transfer`, `api`, `email_notification`, `custom_var`, `folder`.

> **Note**
>
> The API knows two more types, besides the five trigger types: `telestream_qc` (see below) and `antivirus`, which the system creates. The Automation OpenAPI document describes the `data` of all the types.

3.1 File transfer

Transfers files and directories between nodes, workspaces, or cloud buckets using IBM Aspera high-speed FASP protocol.

```yaml
type: transfer
data:
  source:
    type: node                  # node | workspace; in the UI, "Same as trigger" (watch folder and
↪  shared inbox triggers only) leaves node_id and file_id null
    node_id: "95"               # numeric string
    file_id: "12345"
    paths:
      - source: "/"             # path to transfer, relative to file_id
  destination:
    type: workspace
    workspace_id: "638"
    workspace_name: "dev_workspace"
    node_id: "95"
    file_name: "Archive"
    file_id: "67890"
    paths:
      - destination: "{{workflow_instance.uuid}}"  # optional: folder created at the destination when
↪  the workflow runs (supports Sub-Vars); blank = use the destination folder
  rate_policy: high             # high | fair | low | fixed (default fair; the more restrictive of
↪  node and workflow settings applies)
  target_rate:
    rate: 100000                # Kbps (100 Mbps), UI range 10000 to 5000000; the lower of node
↪  setting and this value is used
    allow_retry: true           # true when retry_attempts > 0
    retry_attempts: 3           # 0 to 30; 0 means no retry
    retry_interval: 10          # seconds
  resume_policy:
    resume_partials: true
    difference_check: attrs
    destination_file_options: overwrite_newer # overwrite_always | overwrite_never | overwrite_newer
↪  | overwrite_different | overwrite_both_newer_different
  content_encryption:
```

```
    encryption: aes-256          # none | aes-128 | aes-192 | aes-256; the higher key-length of node
↪   vs. workflow wins
    apply_protection: true       # true when a content protection password is set
    password: "SECRET"           # encrypts the content at the destination; the recipient needs it to
↪   decrypt
  file_attributes: true          # preserve source timestamps at destination
  timeout: 86400000              # milliseconds, default 24 h; the web interface allows 1 h to 48 h,
↪   the API does not check the bounds
  source_archive:                # post-transfer source handling
    archive_option: archive      # none | archive | delete
    move_after_transfer: "/archive"  # archive path, on the same node as the source
```

With `archive_option: delete`, the three boolean options below apply. The web interface sets all three to `true` when `delete` is selected.

```
  source_archive:
    archive_option: delete
    remove_skipped: true             # delete files skipped because they already exist at the
↪   destination
    remove_empty_directories: true   # delete empty nested folders (forced to true by
↪   remove_empty_source_dir)
    remove_empty_source_dir: true    # delete the source folder itself if empty after transfer
```

Note

The archive options are not offered when the source is ”same as trigger”, or when the source is the shared inbox of the trigger. The API rejects the older names `delete_empty_subdirectories` and `overwrite_both_newer_and_different` (it validates `data` strictly). `timeout` is in milliseconds and `target_rate.rate` in Kbps. For a node that Aspera on Cloud does not manage, `source` and `destination` accept `access_key` and `node_url` with an `authentication` object (`node_api` or `ssh`, with base64 `credentials`). `POST /actions/validate` checks these credentials before you create the action. The API encrypts them and never returns them.

When the source is a workspace folder, browsing for the archive folder starts from the workspace home. When the source is a shared folder, the archive folder is selected from the top of that shared folder.

Caution

When `remove_empty_source_dir` is enabled and the workflow has multiple transfer actions, ensure each action uses a **different** source folder to avoid unintended deletions.
`rate_policy` and `target_rate` interact with the node settings: the actual behavior always obeys the **more restrictive** setting.

3.2 Package transfer

Sends digital packages using Aspera Packages application.

```
type: package_transfer
data:
  source:                        # same structure as for the transfer action; "same as trigger" is
↪   allowed
    type: node
    node_id: "95"
    file_id: "12345"
    paths:
      - source: "/"
  source_archive:
    archive_option: none         # none | delete only: archiving the source is not available for this
↪   action
  package:
    workspace_id: "638"
    name: "Automated Package Delivery"
    note: "Package generated by AoC Automation"
```

```
    content_retention_duration: 2592000 # seconds (30 days); the workspace or shared inbox limit
↪   applies, minimum 86400
    # also accepted: bcc_recipients, upload_notification_recipients,
↪   download_notification_recipients,
    # metadata, watermark, encryption_at_rest, delete_package_content_after_download_duration,
↪   send_as_type
    recipients:                 # type is lowercase here: user | group | email | dropbox | newEmail
      - id: "1072"
        type: user
      - id: "58"
        type: group
```

The transfer settings (`target_rate`, `content_encryption.encryption`, `resume_policy.resume_partials`, `file_attributes`, `timeout`) apply as for `transfer`. `rate_policy`, content protection password and `resume_policy.destination_file_options` are not available for this action.

3.3 REST API call

Executes HTTP REST requests against internal or third-party web services with complete payload interpolation.

```
type: api
data:
  method: post                  # get | post | put | delete
  endpoint: "https://api.example.com/v1/ingest"
  authentication:
    type: bearer                # none | bearer | basic (basic uses username and password instead of
↪   token)
    token: "YOUR_API_TOKEN"
  headers:
    - key: "Content-Type"
      value: "application/json"
  payload: '{"job_id": "{{workflow_instance.uuid}}", "file": "{{{step_id_1.action_id_1.filename}}}"}'
↪   # post and put only
  verify_ssl: true
  callback: false               # post and put only; set true to enable API callback (see Configuring
↪   API callback)
  timeout: 300000               # milliseconds; the web interface accepts 1 min (60000) to 24 h
↪   (86400000), default 5 min
```

3.4 Email notification

Dispatches formatted notification emails on workflow milestones.

```
type: email_notification
data:
  send_as_type: User            # the API accepts User only (the workflow owner); Organization is
↪   rejected
  subject: "Workflow Complete: {{{step_id_1.action_id_1.filename}}}"
  message: "The file was successfully processed and moved.\nJob UUID: {{workflow_instance.uuid}}"
  recipient_hashes:           # or email_addresses: a list of addresses; one of the two is required
    - id: "1072"
      type: User                # User | Group | Email; a numeric ID for User and Group, an address
↪   for Email
    - id: "qa_team@example.com"
      type: Email
  timeout: 300000               # milliseconds (default 5 min, 1 to 60 min in the UI)
```

3.5 Folder creation

Dynamically creates directory trees on target nodes prior to file transfer.

```
type: folder
data:
  folder_name: "{{workflow_instance.launched_at.YYYY}}-{{workflow_instance.launched_at.MM}}"
```

```
  parent_folder:
    type: node
    node_id: "95"
    file_id: "12345"
    paths: []                    # required, even when empty
```

3.6 Custom variables

Instantiates named key-value pairs stored in workflow context for downstream consumption. Values can combine literal text, global variables (`{{workflow_instance.uuid}}`), and Sub-Vars from previous steps.

> **Important**
>
> Custom variable names **cannot contain spaces**; the API rejects them.

`custom_variables` is an object whose keys are the variable names. Give each variable its value with a substitution whose `path` targets that key, and always send `substitutions`, even if empty. The action cannot be in step 1.

```
type: custom_var
data:
  custom_variables:
    year: "string"               # placeholder: the value comes from the substitution below
  substitutions:
    - template: "{{workflow_instance.launched_at.YYYY}}"
      path:
        - name: custom_variables
          type: map
        - name: year
          type: map
```

Later actions read the variable as `{{{step_id_<STEP>.action_id_<ACTION>.year}}}`.

3.7 Telestream quality control

Integrates with Telestream cloud services for automated media quality verification. This action is only offered when the `telestreamQc` feature flag is enabled for the organization; it is not part of the public Aspera on Cloud documentation.

```
type: telestream_qc
data:
  endpoint: "https://qc.example.com/api"    # API URL
  api_key: "YOUR_API_KEY"
  body: '{"file": "{{{step_id_1.action_id_1.filename}}}"}'
  timeout: 3600000                # milliseconds (1 to 60 min in the UI; 24 h stored when omitted)
```

3.8 Action timeout defaults

If the Automation app does not receive an update from the node within the configured timeout, the action fails and the job fails.

Action	Default	Minimum	Maximum
`transfer`	24 h	1 h	48 h
`package_transfer`	24 h	1 h	48 h
`api` (no callback)	5 min	1 min	24 h
`api` (with callback)	24 h	1 h	48 h
`email_notification`	5 min	1 min	60 min
`custom_var`	5 min	1 min	60 min
`folder`	5 min	1 min	60 min

3.9 Payload rules enforced by the API

The API validates `data` strictly, and its error messages do not name the missing field. The rules below were verified against the Automation API 1.0.5 through `ascli`: each shape was accepted when the action was created. Creating an action does not run it, so runtime behavior is not covered.

3.9.1 General rules

- Identifiers are strings (`"12345"`). An integer is rejected: `inbox_id of type integer did not match the following type: string`.
- A step accepts actions only once it is listed in the workflow `step_order` (see Creating a workflow step by step).
- Step 1 contains exactly one action, the trigger. Any other action type in step 1 is rejected.
- A location object that carries more properties than its type allows is rejected (`contains additional properties [...]`). A location that lacks a required property is rejected with the unspecific `did not match one or more of the required schemas`: compare it with the table below.
- Optional names are accepted and are what the web interface displays. Add them when you know them.

3.9.2 Location shapes

Used in	Required properties
`shared_inbox` trigger `source`	**All required**: `type: workspace`, `workspace_id`, `workspace_name`, `inbox_id`, `inbox_name`; and `conditions_type` in `data`. Rejected: `dropbox_id`, `node_id`.
`watch_folder` trigger `source`	**Required**: `type: node`, `node_id`, `node_name`, `file_id`, `file_name`, and `paths` with at least one item (`[{source: "/"}]`). `node_name` and `file_name` must both be present. Rejected: `access_key`. The API adds `absolute_path` itself.
`delete_folder` trigger `source`	**Required**: `type: node`, `node_id`, `file_id`, `access_key` (the node access key), and `paths: [{source: "/"}]`. The names are not needed; with the names but without `access_key`, it is rejected.
`transfer` or `package_transfer` `source`, "Same as trigger"	**Required**: `type: workspace`, `workspace_id`, `node_id: null`, `file_id: null`, `paths: []`. `workspace_name` is optional. Omitting the two `null` values is rejected.
`transfer` `source`, explicit	`type: node`, `node_id`, `file_id`, `paths: [{source: "/"}]`. The names are optional.
`transfer` `destination`, node	`type: node`, `node_id`, `file_id`, `paths` (`[]` or `[{destination: "<sub-folder>"}]`). The names are optional.
`transfer` `destination`, workspace	**Required**: `type: workspace`, `workspace_id`, `workspace_name`, `file_name`, `node_id`, `file_id`, `paths: []`. Omitting `workspace_name` or `file_name` is rejected, and so is omitting `node_id` or `paths`; `node_name` is optional.
`folder` `parent_folder`	**Required**: `type: node`, `node_id`, `file_id`, `paths: []`. The names are optional. Omitting `paths` is rejected.

3.9.3 Action-specific rules

Action	Rule
`manual` trigger	`data: {}`.
`scheduled` trigger	`start_date` and `repeat_interval` (`repeat_type`, `repeat_number`), with the limits of Scheduled trigger. `days_of_the_week` is required for `weeks`, `day_of_the_month` for `months`.
`custom_var`	`custom_variables` is an **object** (`{"year": "string"}`), not a list, and `substitutions` must be present (`[]` if unused). The value of a variable comes from a substitution whose `path` is `[{name: custom_variables, type: map}, {name: <variable>, type: map}]`. A list of `{name, value}` pairs is rejected. Not allowed in step 1.

Action	Rule
email_notification	email_addresses or recipient_hashes is required. send_as_type accepts User only (Organization is rejected). recipient_hashes[].type is User, Group or Email, with an address as id for Email.
package_transfer	package.recipients[].type is lowercase: user, group, email, dropbox, newEmail.
api	method and endpoint (an HTTP or HTTPS URL) are required. authentication.type none and bearer (with token) are accepted, and so is callback: true with post. The API masks the token in its response.
workflow_notifications	failed: [{id, type: "User"}] is accepted.

3.9.4 Outputs available to later steps

The API returns the outputs of each action when it is created. Only these fields can be referenced as {{{step_id_<STEP>.action_id_<ACTION>.<FIELD>}}}:

Action	Outputs
scheduled	timestamp
watch_folder, delete_folder	filename, file_id, timestamp, node_id, file_ext, filepath, parent_path
shared_inbox	metadata, name, message, shared_inbox_name, timestamp, dropbox_id, file_id, node_id, sender_name, sender_email, sender_type, sender_id
folder	node_id, file_id, parent_file_id, timestamp
transfer	status, transfer_uuid, timestamp
package_transfer	package_id, status, package_subject, package_message
api	status, response_code, response_body
email_notification	status
custom_var	One output per variable declared in custom_variables
telestream_qc	status, project_id, state, filename, source_url, created_at, updated_at, error_class, error_message
antivirus	status, file_scanned, virus_detected, virus_file_list

Note

When timeout is omitted, the API stores 86400000 (24 h) for transfer, package_transfer, folder, custom_var, telestream_qc and antivirus, and 300000 (5 min) for api (with or without callback) and email_notification. For folder, custom_var, telestream_qc and the api callback, this differs from the web interface defaults in the table above. The API does not check the bounds of the table.

Chapter 4

Substitution variables

Substitution variables (Sub-Vars) let workflow actions consume dynamic values at runtime — from the global job context or from outputs of preceding actions. They use mustache-style double or triple curly braces: `{{global}}` for global variables, `{{{step.action.field}}}` for action outputs.

4.1 Global workflow instance variables

These variables are available in every workflow action regardless of trigger type.

Variable Syntax	Description	Example Value
`{{workflow_instance.uuid}}`	Unique identifier (UUID) of the current job instance.	`a1b2c3d4-e5f6-7890-abcd-ef1234567890`
`{{workflow_instance.launched_at.YYYY}}`	4-digit launch year in UTC.	`2026`
`{{workflow_instance.launched_at.MM}}`	2-digit launch month in UTC.	`11`
`{{workflow_instance.launched_at.MMM}}`	3-letter month name in UTC. The API does not declare it (it declares `Month`); its resolution by the workflow engine is not verified.	`Nov`
`{{workflow_instance.launched_at.DD}}`	2-digit launch day in UTC.	`08`
`{{workflow_instance.launched_at.hh}}`	2-digit launch hour in UTC (24 h).	`14`
`{{workflow_instance.launched_at.mm}}`	2-digit launch minute in UTC.	`30`
`{{workflow_instance.launched_at.date_time}}`	Full ISO 8601 timestamp in UTC (GMT).	`2026-11-08 14:30:00TZD`

Variable Syntax	Description	Example Value
`{{workflow_instance.launched_at.Month}}`	Month. It is the name that the API declares (format not documented).	n/a

4.2 Action output references

Values produced by previous steps and actions are referenced with triple curly braces:

`{{{step_id_<STEP_ID>.action_id_<ACTION_ID>.<FIELD>}}}`

Output Field	Produced By	Description
`filename`	`watch_folder`, `delete_folder`	Base name of the file or folder.
`filepath`	`watch_folder`, `delete_folder`	Full source path of the file.
`parent_path`	`watch_folder`, `delete_folder`	Path to the parent folder.
`file_ext`	`watch_folder`, `delete_folder`	Extension without leading dot (e.g. `mxf`, `mp4`).
`file_id`	`watch_folder`, `delete_folder`, `shared_inbox`, `folder`	Aspera node file identifier.
`node_id`	`watch_folder`, `delete_folder`, `shared_inbox`, `folder`	Node identifier.
`timestamp`	`scheduled`, `watch_folder`, `delete_folder`, `shared_inbox`, `transfer`, `folder`	Time of the event.
`status`	`transfer`, `package_transfer`, `api`, `email_notification`	Status of the action.
`transfer_uuid`	`transfer`	Transfer session UUID.
`response_code`	`api`	HTTP status code (e.g. `200`, `201`).
`response_body`	`api`	HTTP response body.
`package_id`	`package_transfer`	ID of the sent package.

The other outputs are listed in Outputs available to later steps: `metadata`, `shared_inbox_name`, `dropbox_id`, `name`, `sender_name`, `sender_email`, `sender_type`, `sender_id` and `message` for the shared inbox trigger; `package_subject` and `package_message` for `package_transfer`; `parent_file_id` for `folder`. The variables offered for an action are limited to the trigger and actions in the steps above it.

4.3 Path-based substitutions schema

In workflow definitions, substitutions are mapped explicitly using the `substitutions` array in action `data`. Each path element has a `type`: `map` (an object key, with `name`) or `array` (a list, with `name` or `index`). Only `custom_var` also accepts `input_map`, and `folder` accepts `map` only.

```
{
  "substitutions": [
    {
      "template": "Archive/{{{step_id_step1.action_id_act1.filename}}}",
      "path": [
        {"name": "destination", "type": "map"},
        {"name": "paths", "type": "array"},
        {"index": 0, "type": "map"},
        {"name": "destination", "type": "map"}
      ]
    }
  ]
}
```

Chapter 5

Managing workflows and jobs

The **Workflows** page lists all your workflows and their current state. The **Job Queue** page shows all running and queued jobs across your workflows. The actions below are available from the overflow menu (⋮) on each row.

5.1 Workflow lifecycle

From the overflow menu (⋮) next to a workflow name in the Workflows page:

Action	Effect	New Status
Activate	Workflow starts launching jobs automatically (event triggers) or on **Run Now** (manual). API: `update_state` with `event: activate`. From `draft`, the workflow first goes to `validating`, then to `active` when the permissions on the nodes are granted. The API refuses a workflow whose status is still `not_started`, and a `scheduled` trigger whose `start_date` is in the past.	`active`
Deactivate	No new job is launched; running and queued jobs complete (or can be canceled). Org admins can deactivate (and activate) other users' workflows. API: `event: deactivate`.	`inactive`
Edit	Puts the workflow back into edit mode (`event: save_as_draft`). Only possible when the workflow has no executing, initiating or queued job and is not `validating`; an active workflow is deactivated first, after confirmation.	`draft`
Duplicate	Creates a copy in `draft` state. Org admins can duplicate other users' workflows. The copy may fail at run time if you do not have access to the resources it references (shared inbox, shared folder). API: `POST /workflows/:id/copy` with the mandatory `name` of the copy (`GET /workflows/:id/default_copy_name` proposes one); the API refuses while the workflow status is `submitted`.	`draft`
Delete	Removes the workflow from the list. The API refuses while a job is executing, initiating or queued: cancel the jobs first (`PUT /workflows/:id/cancel_instances`).	—
Run Now	Manually launches one job instance. The workflow must be `active`, with a `manual` or `scheduled` trigger: the API refuses `watch_folder`, `shared_inbox` and `delete_folder` workflows (422).	job `running`
Share	Grants another user the **Manage jobs** permission (view, start, cancel, delete jobs). The API also has the `duplicate` and `edit` permissions (`edit` implies the others in the checks). The permission of the owner cannot be changed.	no change
Rename	Renames the workflow; no operational effect.	no change
Email notifications	Configures per-event email alerts (started, completed, failed) for the workflow.	no change
Job queue	Jumps to the job queue filtered to this workflow.	no change

> **Note**
>
> Org admins can deactivate and duplicate workflows created by other users.

Listing workflows (`GET /workflows`) accepts the query parameters `q` (filter on name, state, status and owner), `sort` (column; a `-` prefix sorts in descending order), `show_org_workflows` (all non-hidden organization workflows instead of your own, for an administrator), `response_type` (`short`, default, or `long` to include the full steps and actions), `offset` and `limit`. Listing workflow instances (`GET /workflow_instances`) accepts `workflow_id`, `status`, `q`, `sort`, `show_org_workflow_instances` (administrator), `start_time` and `stop_time` (both required to filter on the last update time), `embed` (`workflow`, `step`), `offset` and `limit`. `user_id` and `organization_id` come from the bearer token.

5.1.1 API access and pagination

- The bearer token must be a valid JSON Web Token (otherwise `401`) that gives access to the Automation app, which must be enabled for the organization (otherwise `403`). Outside the administrator scope, the caller must hold a permission on the workflow (`403` otherwise); an unknown ID returns `404`.
- `edit` is required to change a workflow, its steps, actions and permissions; `manage_jobs` (or `edit`) to launch, cancel and delete jobs; `duplicate` (or `edit`) to copy.
- Deleting a workflow, a step or an action returns `422` while a job is executing, initiating or queued.
- Lists return `total_count`, `offset`, `limit` and the links `first`, `last`, `next`, `previous`. `offset` is a number of records to skip, not a page number. `limit` is 20 by default and 1000 at most.

5.1.2 Automatic deactivation after failures

The organization-level Automation settings are read with `GET /app_permissions/:organization_id` and updated with `PUT /app_permissions/:organization_id`; the ID must be that of the organization of the token, and the update requires the administrator scope. The only setting you can change is `failed_count_to_deactivate` (default 10): the number of consecutive failed runs of a workflow before it is deactivated automatically. The response also has `enabled` and `available`.

5.2 Job lifecycle

From the **Job Queue** page (check the job row, then click the action):

Action	Effect	New Status
Cancel running jobs	Sends an async cancel to the node for in-progress transfers; reports job as `canceled`. All queued jobs are canceled automatically.	`canceled` (displayed ”Cancelled”)
Delete jobs	Removes the job from the list.	—

Job step indicators: **green** = success, **blue** = in progress, **red** = failed.

Chapter 6

Managing workflows with ascli

`ascli` covers the full Automation lifecycle: CRUD on workflows, state transitions, job management, export/import for version control, and permissions. All commands start with `ascli aoc automation`.

6.1 CLI commands reference

`ascli` provides full command-line lifecycle management for AoC Automation:

```
# Workflows CRUD and operations
ascli aoc automation workflows list [--query.show_org_workflows=@val:true]
ascli aoc automation workflows show <workflow_id>
ascli aoc automation workflows details <workflow_id>
ascli aoc automation workflows create @json:'{"name":"My Workflow"}'
ascli aoc automation workflows update_state <workflow_id> @json:'{"event":"activate"}'   # activate |
 ↪  deactivate | save_as_draft
ascli aoc automation workflows delete <workflow_id>
ascli aoc automation workflows duplicate <workflow_id> [--name="New Name"]
ascli aoc automation workflows export <workflow_id>
ascli aoc automation workflows import @yaml:@file:my_workflow.yml

# Execution and instances
ascli aoc automation workflows launch <workflow_id>
ascli aoc automation workflows cancel_instances <workflow_id>
ascli aoc automation instances list [--query.workflow_id=<workflow_id>]
ascli aoc automation instances show <instance_id>
ascli aoc automation instances cancel <instance_id>

# Permissions and sharing
ascli aoc automation workflows permissions <workflow_id> list
ascli aoc automation workflows permissions <workflow_id> create
 ↪  @json:'{"user_id":"...","manage_jobs":true}'
```

6.2 Exporting and importing workflows

Workflows can be fully exported to portable YAML/JSON files for Git version control and CI/CD promotion across environments.

6.2.1 Exporting a workflow

```
ascli aoc automation workflows export 1120 --format=yaml --out.file=workflow_backup.yml
```

6.2.2 Portable manifest format example

```
name: "Ingest and Distribute Video"
trigger_type: "watch_folder"
allow_notifications: true
```

```yaml
workflow_notifications:
  failed:
    - id: "admin@example.com"
      type: "Email"
steps:
  - ref_id: "step_1"
    name: "Watch Drop Folder"
    actions:
      - ref_id: "act_trigger"
        name: "Watch Drop Folder"
        type: "watch_folder"
        data:
          source:
            type: "node"
            node_id: "95"
            file_id: "100"
          conditions:
            - field_name: "extension_name"
              field_type: "string"
              match_type: "matches"
              value: "mp4"
  - ref_id: "step_2"
    name: "Transfer to Cloud Archive"
    actions:
      - ref_id: "act_transfer"
        name: "FASP Transfer"
        type: "transfer"
        data:
          source:
            type: "node"
            node_id: "95"
            file_id: "100"
          destination:
            type: "workspace"
            workspace_id: "638"
            file_id: "200"
          resume_policy:
            destination_file_options: "overwrite_newer"
          substitutions:
            - template: "Archives/{{{step_id_step_1.action_id_act_trigger.filename}}}"
              path:
                - name: "destination"
                  type: "map"
                - name: "path"
                  type: "map"
```

6.2.3 Importing a workflow

```
ascli aoc automation workflows import @yaml:@file:workflow_backup.yml
```

`@yaml:` expects inline YAML: to read a file, chain it with `@file:`.

`ascli` creates the workflow, then each step in order. It adds the step to the workflow `step_order` before creating its actions, because the API refuses an action whose step is not in the workflow. In every string of the action data, it replaces the `ref_id` of steps and actions by the identifiers it created. The workflow is created in `draft` state. If a call fails, `ascli` deletes the partial workflow and reports the error.

> **Note**
>
> Before `ascli` 4.29, `import` and `duplicate` created all steps first, failed on a manifest with a step after the trigger, left a partial `draft` workflow behind, and replaced identifiers only in `substitutions[].template`.

6.2.4 Creating a workflow step by step

To build or fix a workflow one call at a time:

```
ascli aoc automation workflows create @json:'{"name":"My Workflow"}'
 ↪  # returns <WF>
ascli aoc automation steps create @json:'{"workflow_id":"<WF>","name":"Trigger"}'
 ↪  # returns <STEP1>
ascli aoc automation workflows modify <WF> @json:'{"step_order":["<STEP1>"]}'
 ↪  # before creating actions
ascli aoc automation actions create @json:'{"step_id":"<STEP1>","type":"manual","data":{}}'
ascli aoc automation steps create @json:'{"workflow_id":"<WF>","name":"Next"}'
 ↪  # returns <STEP2>
ascli aoc automation workflows modify <WF> @json:'{"step_order":["<STEP1>","<STEP2>"]}'
ascli aoc automation actions create @json:'{"step_id":"<STEP2>","type":"transfer","data":{...}}'
ascli aoc automation workflows details <WF>
 ↪  # check the result
ascli aoc automation workflows delete <WF>
 ↪  # discard a failed attempt
```

Note

A step created through `steps create` is only listed in the workflow `step_order` after `workflows modify`.

6.3 Finding the identifiers

The API requires IDs and names in every location. Collect them first, with the preset of the organization (`--preset=<PRESET>`):

To find	Command	Fields to keep
Your user ID, for recipients	`ascli aoc --preset=<PRESET> user profile show --fields=id,email`	`id`
Default workspace	`ascli aoc --preset=<PRESET> user workspaces current`	`id`, `name`, `home_node_id`, `home_file_id`
All workspaces (node and name)	`ascli aoc --preset=<PRESET> admin workspace list --fields=id,name,node_id`	`id`, `name`, `node_id`
Node name and access key	`ascli aoc --preset=<PRESET> admin node show <NODE_ID> --fields=id,name,access_key,host`	`name`, `access_key`
Folder IDs in the workspace	`ascli aoc --preset=<PRESET> files browse / --fields=name,id,type`	`id`, `name` of the entries of type `folder`
Shared inboxes	`ascli aoc --preset=<PRESET> packages shared_inboxes list --fields=ALL --format=json`	`dropbox_id` (use as `inbox_id`), `dropbox.name`, `workspace_id`, `dropbox.metadata_schema`
Existing workflows	`ascli aoc --preset=<PRESET> automation workflows list`	`id`, `name`, `state`

In the output of `files browse`, entries of type `link` are shared folders and entries of type `folder` are regular folders. Use the `id` of an entry of type `folder` as `file_id`.

Chapter 7

Configuring API callback

When `callback: true` is set on an `api` action, after sending its HTTP request the workflow **pauses** and waits for the external service to call back.

Prerequisites — your API server must: 1. Include the **static callback endpoint** `https://api.ibmaspera.com/automation/v1/external/api_callback` in its response. 2. Echo back the **token** that AoC injects into the payload (parse it from the incoming request body). The callback request is not authenticated; `token` and `status` are mandatory (otherwise `422`).

Enabling in the UI: 1. Open or create a workflow → select the **Call an API** action. 2. Set **Request type** to `POST` or `PUT`. 3. Check **Add an API callback**. 4. Save and activate the workflow.

Sequence of events (once the action is triggered):

1. AoC sends the configured JSON payload to your API, with a unique `token` field injected automatically. The workflow pauses (default timeout in the web interface: **24 h**; the API stores 5 min if `timeout` is omitted).
2. *(Optional)* Your service may send **in-progress** status notifications to the callback endpoint:

   ```
   { "token": "<TOKEN>", "status": "in_progress", "progress": 42 }
   ```

   Progress (0–100) is displayed in the Automation UI.
3. When your service finishes, it sends a **completion** response:

   ```
   { "token": "<TOKEN>", "status": "success" }
   ```

   or `"status": "failed"` to fail the job.
4. On `success`, the workflow proceeds to the next step.

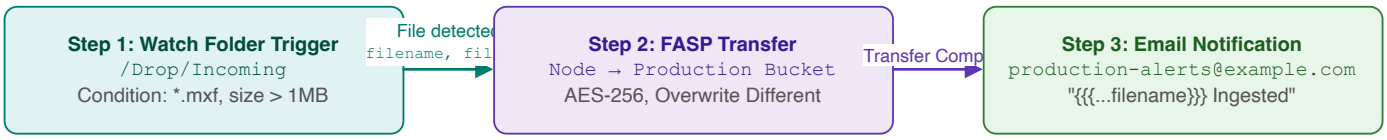
Chapter 8

Design recipes and patterns

Common end-to-end workflow patterns combining triggers, transfers, API calls, and notifications. Each recipe is illustrated with a diagram showing the data flow.

8.1 Watch folder to multi-destination transfer

Goal: Automatically ingest incoming media files, replicate them to production storage with AES-256 encryption, and notify the production team.



Watch folder trigger to multi-destination transfer and notification

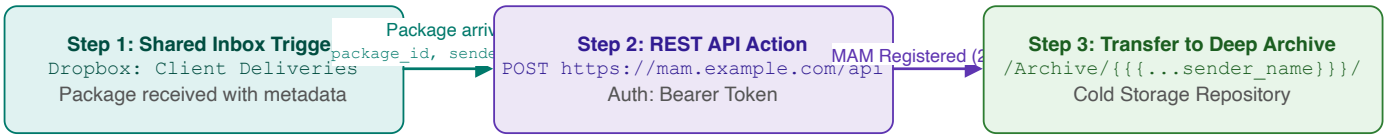
---

8.2 Shared inbox package to MAM webhook

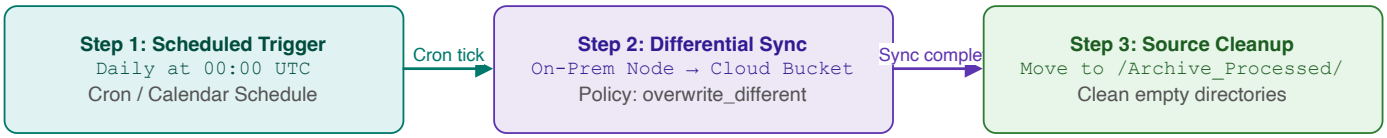
Goal: Listen for client deliveries in a Shared Inbox (Dropbox), notify an external MAM system via REST API, and transfer the payload into deep archive.

8.3 Scheduled sync with differential overwrite

Goal: Run a nightly differential sync every midnight from an on-premise node to cloud storage, then move processed files to an archive folder.



Shared inbox package reception to MAM webhook and deep archive



Scheduled sync with differential overwrite and archive cleanup

8.4 Shared inbox fan-out with archive, webhook, review package and summary

Goal: When a partner delivers a package, create a dated folder. Then, in parallel, archive the content with AES-256, notify an external system, and forward the content as a package for review. Finally, send a summary e-mail that reports the outputs of the three parallel actions.

```
Step 1  shared_inbox        Package received
Step 2  folder              <YYYY>-<MM>-<DD>_<sender_name> in the delivery folder
Step 3  transfer            AES-256 archive copy          (parallel)
        api                 POST to the external webhook  (parallel)
        package_transfer    review package                (parallel)
Step 4  email_notification  summary with response_code, transfer_uuid and package_id
```

Manifest, in the format of `export` (see Portable manifest format example). Replace the `<PLACEHOLDER>` values with those found in Finding the identifiers. The `substitutions` arrays, which repeat the templates of the action fields, are omitted for brevity: add one for every field that contains a Sub-Var.

```yaml
name: "Media Delivery Pipeline"
trigger_type: "shared_inbox"
allow_notifications: true
workflow_notifications:
  failed:
    - id: "<USER_ID>"
      type: "User"
steps:
  - ref_id: "trigger"
    name: "Package received"
    actions:
      - ref_id: "inbox"
        type: "shared_inbox"
        data:
          source:
            type: "workspace"
            workspace_id: "<WORKSPACE_ID>"
            workspace_name: "<WORKSPACE_NAME>"
            inbox_id: "<INBOX_ID>"
            inbox_name: "<INBOX_NAME>"
          conditions_type: "all_new"
  - ref_id: "prepare"
    name: "Prepare destination"
    actions:
      - ref_id: "folder"
        type: "folder"
        data:
          folder_name: "{{workflow_instance.launched_at.YYYY}}-{{workflow_instance.launched_at.MM}}-
↪  {{workflow_instance.launched_at.DD}}_{{{step_id_trigger.action_id_inbox.sender_name}}}"
          parent_folder:
            type: "node"
            node_id: "<NODE_ID>"
            file_id: "<FOLDER_ID>"
            paths: []
  - ref_id: "fanout"
    name: "Archive, notify, forward for review"
    actions:
      - ref_id: "archive"
        type: "transfer"
        data:
          source:                          # "Same as trigger"
            type: "workspace"
            workspace_id: "<WORKSPACE_ID>"
            node_id: null
            file_id: null
            paths: []
          destination:
            type: "node"
            node_id: "<NODE_ID>"
            file_id: "<FOLDER_ID>"
```

```
          paths: []
        content_encryption:
          encryption: "aes-256"
        resume_policy:
          destination_file_options: "overwrite_newer"
    - ref_id: "webhook"
      type: "api"
      data:
        method: "post"
        endpoint: "<WEBHOOK_URL>"
        authentication:
          type: "none"
        payload: '{"job_id": "{{workflow_instance.uuid}}", "sender":
↪  "{{{step_id_trigger.action_id_inbox.sender_email}}}"}'
    - ref_id: "review"
      type: "package_transfer"
      data:
        source:                         # "Same as trigger"
          type: "workspace"
          workspace_id: "<WORKSPACE_ID>"
          node_id: null
          file_id: null
          paths: []
        source_archive:
          archive_option: "none"
        package:
          workspace_id: "<WORKSPACE_ID>"
          name: "Review: {{{step_id_trigger.action_id_inbox.name}}}"
          recipients:
            - id: "<USER_ID>"
              type: "user"              # lowercase for package_transfer
  - ref_id: "notify"
    name: "Summary notification"
    actions:
      - ref_id: "mail"
        type: "email_notification"
        data:
          send_as_type: "User"
          subject: "Delivery processed: {{{step_id_trigger.action_id_inbox.name}}}"
          message: "Webhook status: {{{step_id_fanout.action_id_webhook.response_code}}}\nArchive
↪  transfer: {{{step_id_fanout.action_id_archive.transfer_uuid}}}\nReview package:
↪  {{{step_id_fanout.action_id_review.package_id}}}"
          recipient_hashes:
            - id: "<USER_ID>"
              type: "User"              # capitalized for email_notification
```

Import it with `ascli aoc automation workflows import @yaml:@file:media-delivery-pipeline.yml`. The workflow is created in `draft` state.

> **Note**
>
> The API validates the manifest when the workflow is created. The runtime resolution of its Sub-Vars is not verified until a job runs.

Chapter 9

Best practices and troubleshooting

Guidelines to build reliable production workflows, and a table of common failure modes with their root causes and resolutions.

9.1 Practices checklist

- [ ] **Trigger Isolation**: Keep the trigger step dedicated solely to event capture. Perform heavy transfer or API operations in subsequent steps.
- [ ] **Prevent Trigger Loops**: When configuring a `watch_folder` trigger with destination on the same node, configure `source_archive` (archive or delete) on the `transfer` action, or ensure destination paths are excluded from watch rules. The `source_archive` setting belongs to the transfer action, not to the trigger.
- [ ] **Distinct Source and Destination**: The API refuses a `transfer` action whose source and destination are the same (`'source' and 'destination' cannot be the same for transfer step`).
- [ ] **Transfer Idempotency**: Use `overwrite_newer` or `overwrite_different` rather than `overwrite_always` to prevent unnecessary bandwidth consumption during retries.
- [ ] **Secrets Handling**: Do not hardcode static credentials in API action bodies. Use bearer tokens or scoped access keys.
- [ ] **Error Notifications**: Always define `workflow_notifications.failed` recipients on production workflows so the operations team is alerted immediately on failures.
- [ ] **Validate in Draft First**: Deploy new workflows in `draft` and check them with `ascli aoc automation workflows details <ID>` before activating. A workflow in `draft` never launches jobs.
- [ ] **Resolve Identifiers Before Writing**: Look up every node, folder, workspace and shared inbox ID (and its name) with the commands in Finding the identifiers, because the API requires the names as well as the IDs.

9.2 Common pitfalls and resolutions

Issue	Root Cause	Resolution
Workflow cannot be activated	Missing trigger action, or fewer than two steps, or a step with no actions, or `manual`/`scheduled` trigger with no transfer action, or the permissions on the nodes not yet granted (status `not_started`, `submitted` or `error`), or a `scheduled` trigger whose `start_date` is in the past.	Ensure Step 1 has a valid trigger; the workflow has at least two steps and every step has at least one action; `manual` and `scheduled` triggers include a `transfer` or `package_transfer` action; update `start_date`.
422 on `launch`	The workflow is not `active`; or its trigger is `watch_folder`, `shared_inbox` or `delete_folder`; or it is `scheduled` and already has a running job.	Activate the workflow; these triggers fire on their own event.
422 on delete of a workflow, step or action	A job is executing, initiating or queued.	Cancel the jobs (`cancel_instances`), then retry.
Cannot edit workflow	A workflow can only be made editable (`save_as_draft`) when it has no running jobs.	Run `ascli aoc automation workflows cancel_instances <id>`, or **Deactivate** from the UI and wait for the running and queued jobs to complete, then **Edit**.
Substitution variable not resolving	Target step or action ID mismatch, or referencing a downstream step.	Verify that Sub-Vars only reference steps positioned **above** the current step in `step_order`.
Transfer action fails on permission	Aspera on Cloud service user lacks write permissions on destination node.	Ensure the node access key configured in AoC has read/write privileges on the destination storage path.
Transfer files are not re-transferred	Files already present at the destination are skipped (resume policy).	Use `overwrite_always` to force re-transfer, or `overwrite_newer`/`overwrite_different` for differential sync.
Cancel does not stop the node transfer	Automation sends an async cancel; the node may not honour it immediately.	The job is reported `canceled` by Automation even if the underlying FASP session is still active on the node. Monitor the node logs if immediate cancellation is critical.
Job stays in `queued` indefinitely	Concurrent job maximum reached (5 Standard Plus, 10 Premium).	Wait for running jobs to complete, cancel them from the Job Queue, or upgrade the subscription edition.

Issue	Root Cause	Resolution
API callback never received	External server not configured to call back, or wrong token.	Verify that the server POSTs to `https://api.ibmaspera.com/automation/v1/external/api_callback` with the original `token` value and `"status": "success"` or `"failed"`.
`422 ... did not match one or more of the required schemas` on `#/source`, `#/destination` or `#/parent_folder`	The location object lacks a field the API requires: a name (`node_name`, `file_name`, `workspace_name`, `inbox_name`) or `paths`. The message never says which one.	Copy the exact shapes from Payload rules enforced by the API.
`422 ... contains additional properties ["dropbox_id"]` (or `node_id`)	A property that is not allowed for this location type.	Remove it: a `shared_inbox` source uses `inbox_id`, not `dropbox_id`; a `workspace` trigger source has no `node_id`.
`422 ... Step Step for this action doesn't belong to a workflow`	The step is not yet in the workflow `step_order`. The API appends this message to every other validation error on the same call, so it is often a side effect.	Add the step ID to `step_order` right after creating the step, before creating its actions. Read the rest of the message first.
`422 ... First step of the workflow should only have one action and that should be a trigger`	A non-trigger action was created in step 1.	Keep the trigger alone in step 1; create the other actions in step 2 or later.
`422 ... value "Organization" did not match` on `send_as_type`	Only `User` is accepted.	Use `send_as_type: User`, or omit it.
`422 ... recipients/0/type value "User" did not match`	The recipient type is case-sensitive and differs per action.	`package_transfer`: `user`, `group`, `email`, `dropbox`, `newEmail`. `email_notification` and `workflow_notifications`: `User`, `Group`, `Email`.
`ascli aoc automation workflows import` fails with "Step ... doesn't belong to a workflow"	`ascli` before 4.29 created all steps, then set `step_order` once at the end, and left a partial workflow.	Upgrade `ascli`, or use the manual sequence.
`ascli ... import @yaml:file.yml` fails with "Argument workflow (Hash) is a String"	`@yaml:` expects inline YAML, not a file name.	Use `@yaml:@file:file.yml`.

IBM Aspera®

End of document